
SoC Motor Driver

1.0 SERIAL COMMUNICATION INTERFACE (SCI) UNIT

1.1 Features

- Two independent channels
- Full support of SPI communication
- Full support of UART communication
- Independent interrupt request sources for each channel
- DMA flow-control interface for each channel

1.2 Introduction

The device features two SCI units dedicated to handle all serial communication with external devices. Each channel can be independently configured for SPI communication or UART communication. Each channel has a dedicated interrupt request signal and a DMA flow-control interface. The SCI units use the GPIO terminal 0 to 7 for the connections to external devices.

SoC Motor Driver

1.3 Functional Description

Each SCI unit consists of SPI and UART modules to handle all serial communication. There are two SCI units: SCI0 and SCI1. Each unit can be configured independently of the other channel. In other words, the user may set up one channel for SPI communication and the other for UART communication.

To configure the SCI unit, the corresponding SCIx_MODE signal is programmed via the sysctrl_control register in the System Control Unit. This signal configures the SCI unit and its I/Os as SPI or UART blocks. Table 1 shows the different settings of SCIx_MODE.

Table 1: SCIx modes of operation

Signal	Value	Description
SCIx_MODE	0	SCIx operates as an SPI module
	1	SCIx operates as a UART module

The serial communication signals SCIx_RX and SCIx_TX are imported and exported, respectively, via the GPIO pins. The SPI and UART Rx inputs (SPiX_RX and UARTx_RX respectively) are selected from the 8 GPIO pins via the corresponding sysctrl_sel_spix and sysctrl_sel_uartx registers, in the System Control Unit. These registers are used to select which input on the GPIO pins is used as the input to the corresponding SPiX or UARTx channel. For more information, refer to the System Control Unit section.

The SCIx_TX signals are exported to GPIO. Each SCI channel's Tx output is connected to all GPIO pins. The user shall select the desired output from the output source mux of the desired GPIO pin via its corresponding gpio_controlx register. For more information, refer to the GPIO section.

In case of using the SPI module as a slave node, the input serial clock (SPiX_SCKIN) and chip-select (SPiX_CSIN), imported via GPIO pins, are fed through directly to the SPI module. The input source of these signals is configurable via the sysctrl_sel_spix register in System Control Unit.

Each SCI channel supports DMA transfers by incorporating a DMA flow-control interface. The SPI and UART modules output TX_READY and RX_READY signals. These signals are multiplexed and the selected signal is routed to the DMA Controller. The output of the multiplexer is selected via the SCIx_MODE signal. Similarly, the TX_ACK and RX_ACK signals are input to the SCI unit. The input to the SCIx channel is de-multiplexed and routed to the corresponding block (SPI or UART) via the SCIx_MODE signal.

Finally, each SCIx channel has its dedicated, active-high IRQ_SCIx signal connected to the Cortex M4's NVIC. Each channel's SPI and UART generate an interrupt request signal (IRQ_SPiX and IRQ_UARTx respectively). These signals are then multiplexed forming the single output IRQ_SCIx. The output is selected via the SCIx_MODE signal.

Table 2 summarizes the SCI input signals, and a description of their configuration.

Table 2: Summary of SCI input signals

Input Signal	Register	Description
scix_mode	sysctrl_control register in System Control Unit	This signal configures the corresponding SCI channel to operate either as an SPI module or a UART module. SCI Tx and Rx signals, SCI DMA flow-control inputs and outputs, as well as the SCI interrupt requests are routed via this signal. This signal also selects whether the APB bus is connected to the SPI or UART module.
spix_sckin	The GPI source is selected via the sckin[3:0] bits in the sysctrl_sel_spix register in System Control Unit	Input serial clock source imported from a GPIO pin for SPI communication.
spix_csin	The GPI source is selected via the csin[3:0] bits in the sysctrl_sel_spix register in System Control Unit	Input chip-select signal imported from a GPIO pin for SPI communication.

SoC Motor Driver

sel_uartx_rx	sysctrl_sel_uartx register in System Control Unit	This signal selects which GPI signal is used as the serial input Rx for the UART module.
sel_spix_rx	sysctrl_sel_spix register in System Control Unit	This signal selects which GPI signal is used as the serial input Rx for the SPI module.
scix_rx_ack	-	DMA flow-control signal. Routed to either UART or SPI module via the scix_mode signal.
scix_tx_ack	-	DMA flow-control signal. Routed to either UART or SPI module via the scix_mode signal.

Figure 1-1 shows the general block diagram of an SCI channel. Both SCI channels inherit the shown architecture.

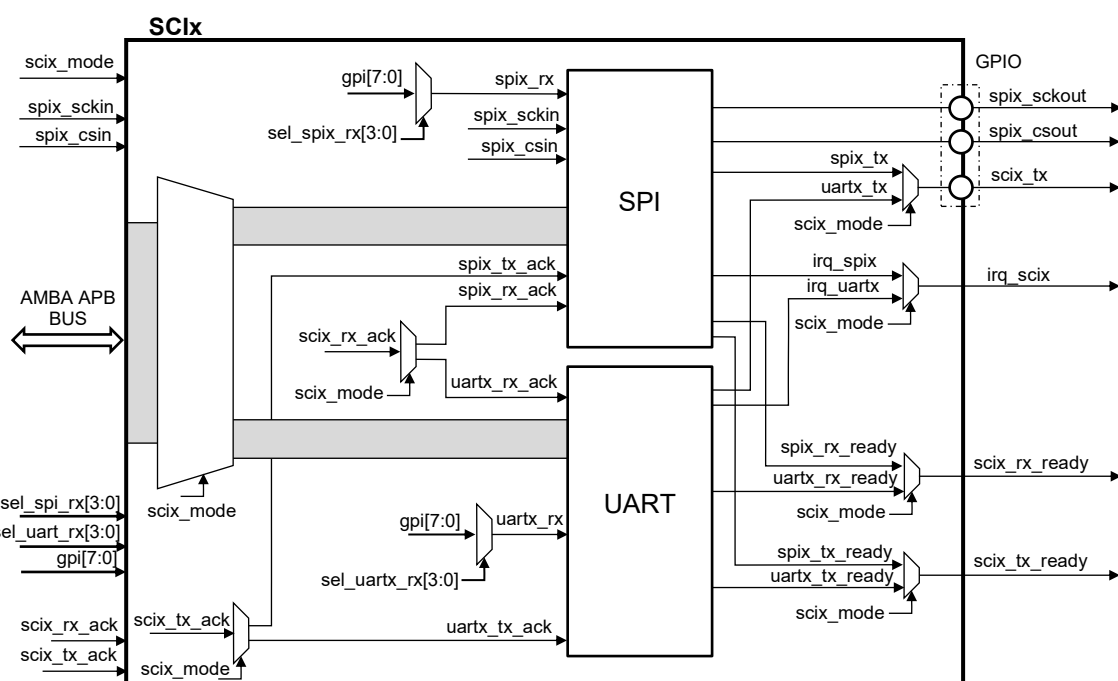


Figure 1-1: General block diagram of an SCI channel

1.4 APB Bus Access

For a given SCI channel, the SPI and UART module share the same APB address, since both modules are mutually exclusive and cannot be used at the same. Selecting the desired module via the SCIX_MODE signal in the sysctrl_control register will route the APB bus interface to the desired module's internal register set.

SCI0 Base Address: 0x4000A000

SCI1 Base Address: 0x4000B000

SoC Motor Driver

1.5 DMA Interface

Each SCI module features a DMA flow controller and an interface to support requests between the module and the DMA controller. The DMA interface can be used such that the DMA module can be configured to automatically transfer data in either direction between the SCI and memory or other peripherals. The signals interfacing the SCI, the DMA flow controller, and the DMA module are handled automatically.

The DMA flow controller generates two output signals: SPIX_RX_READY and SPIX_TX_READY, which indicate that the SPI module has data to be read, or that it is ready to accept new data, respectively. On the other hand, the DMA flow controller receives two input signals which are controlled by the DMA Controller: SPIX_RX_ACK and SPIX_TX_ACK, which acknowledge any request from the SPI module after the transfer is complete, to avoid any overflow or underrun.

Table 3 summarizes the DMA request signals and their drive sources. The asserted signals are automatically de-asserted when the DMA Controller asserts the corresponding acknowledge signal after the transfer is completed.

Table 3: DMA request signals

Signal	Drive Source	Description
SPIX_RX_READY	RXAF	This signal is set when the receive FIFO almost full flag RXAF is set. This flag is set when the number of entries in the Rx FIFO exceeds the programmed threshold value stored in the spix_rxaf_threshold register.
SPIX_TX_READY	TXAE	This signal is set when the transmit FIFO almost empty flag TXAE is set. This flag is set when the number of entries in the Tx FIFO is less than the programmed threshold value stored in the spix_txae_threshold register.

1.6 Serial Peripheral Interface (SPI) Communication

1.6.1 Features

- Full-duplex, synchronous serial communication
- Two independent channels
- Master or slave operation
- 16-message deep Rx and Tx FIFO buffers
- Programmable word size (1 to 16 bits)
- Programmable bit ordering (MSb first/ LSb first)
- Programmable clock polarity and phase
- Programmable bit rate
- Automatic and manual chip-select control
- Auto Tx and Rx to reduce bus bandwidth requirements
- DMA flow-control interface
- Configurable interrupt request sources

1.6.2 Introduction

The device features an SPI module with two independent channels for full-duplex, synchronous serial communications. The module can be configured as a master or slave and provides flexible configurations for word size, bit ordering, clock polarity and phase. The module also hosts a DMA flow-control interface to support DMA transfers.

SoC Motor Driver

1.6.3 Functional Description

The device features two independent SPI channels in the SCI unit. Each channel can be configured either as an SPI slave or an SPI master.

When configured as a slave, the module uses an input chip select and serial clock signals, which are imported via the GPIO pins. For more information on configuring input chip select and serial clock signals, refer to the System Control Unit section.

When the module is configured as a master, it generates the serial clock signal SPIx_SCKOUT and the chip select signal SPIx_CSOUT for a single slave bus¹. All signals are imported or exported via GPIO.

The user software is able to read or write to the Rx and Tx FIFO buffers, respectively, via the corresponding spix_rx and spix_tx registers. The Rx and Tx FSMs handle data transmissions and set any necessary status bits. Depending on the interrupt generation configuration, the interrupt management system will generate a single interrupt request signal IRQ_SPIx to the Cortex M4's NVIC. The DMA flow controller will handle the handshake with the DMA controller to request DMA transfers.

The serial clock controller logic will generate the SPIx_SCKOUT serial clock signal. The output clock frequency, and the clock sampling phase are programmable. Clock polarity settings are configurable as well.

Figure 1-2 shows the general block diagram of a given SPI channel.

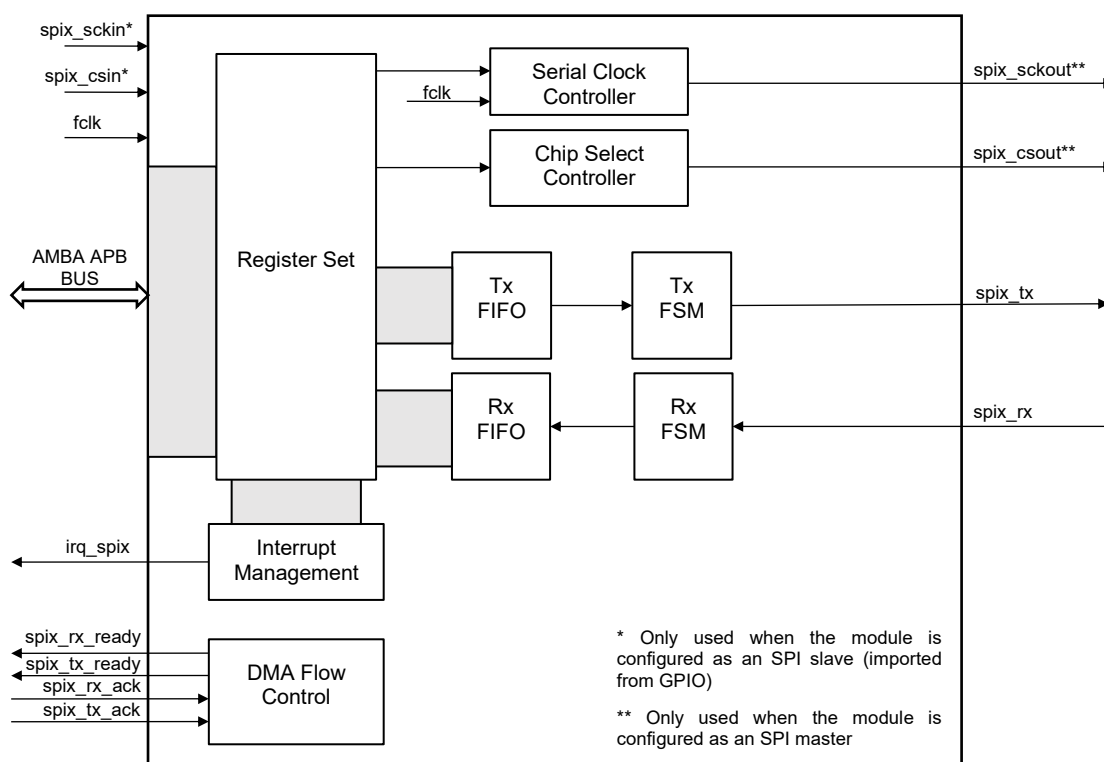


Figure 1-2: General block diagram of an SPI channel

¹ For multiple slave buses, software-controlled GPIO output signals can be used for multiple chip select outputs.

SoC Motor Driver

1.6.4 Register Set

The SPI register set is accessible through the AMBA APB Bus interface. Registers are word-aligned. Reserved and unimplemented bits are read as zero.

The register map consists of 16 16-bit registers. Each channel has its dedicated register set. Table 4 shows the register map of SPI channels 0 and 1 and summarizes the registers' names, access type, address offset and reset values.

SPI0 Base Address: 0x4000A000

SPI1 Base Address: 0x4000B000

Table 4: SPI register map

Register Name	Access Type	Address Offset	Reset Value
SPI0 Base Address: 0x4000A000			
spi0_tx	W	0x00	0x0000
spi0_rx	R	0x04	0x0000
spi0_status	R/1C	0x08	0x4011
spi0_control	R/W	0x0C	0x0000
spi0_cycles_per_bit	R/W	0x10	0x0002
spi0_txae_threshold	R/W	0x14	0x0008
spi0_rxaf_threshold	R/W	0x18	0x0008
spi0_tx_count	R	0x1C	0x0000
spi0_rx_count	R	0x20	0x0000
spi0_autotrx_control	R/W	0x24	0x0000
spi0_autotrx_count	R/W	0x28	0x0000
spi0_cs_control	R/W	0x2C	0x0000
spi0_cs_assert	R/W	0x30	0x0001
spi0_cs_assert_bits	R/W	0x34	0x0001
spi0_cs_deassert_bits	R/W	0x38	0x0001
spi0_cs_idle_bits	R/W	0x3C	0x0001
SPI1 Base Address: 0x4000B000			
spi1_tx	W	0x00	0x0000
spi1_rx	R	0x04	0x0000
spi1_status	R/1C	0x08	0x4011
spi1_control	R/W	0x0C	0x0000
spi1_cycles_per_bit	R/W	0x10	0x0002
spi1_txae_threshold	R/W	0x14	0x0008
spi1_rxaf_threshold	R/W	0x18	0x0008
spi1_tx_count	R	0x1C	0x0000
spi1_rx_count	R	0x20	0x0000
spi1_autotrx_control	R/W	0x24	0x0000
spi1_autotrx_count	R/W	0x28	0x0000
spi1_cs_control	R/W	0x2C	0x0000
spi1_cs_assert	R/W	0x30	0x0001
spi1_cs_assert_bits	R/W	0x34	0x0001
spi1_cs_deassert_bits	R/W	0x38	0x0001
spi1_cs_idle_bits	R/W	0x3C	0x0001

SoC Motor Driver

spix_tx

Reset value = 0x0000

Address Offsets - 0x00

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
DATA15	DATA14	DATA13	DATA12	DATA11	DATA10	DATA9	DATA8
W	W	W	W	W	W	W	W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
DATA7	DATA6	DATA5	DATA4	DATA3	DATA2	DATA1	DATA0
W	W	W	W	W	W	W	W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
DATA<15:0>	15:0	Data to be transmitted over the serial interface. Data written to this register will be written to the Tx FIFO. To avoid any data loss, this register should not be written to if the TXF bit in the spix_status register is set. The number of transmitted bits is determined by the BPW bits in the spix_control register. If the module is configured as an SPI master, the transmission will begin once this register is written. If the module is configured as an SPI slave, this register must be written before the SPI master drives the serial clock signal, in order to transmit data.

SoC Motor Driver

spix_rx

Reset value = 0x0000

Address Offsets - 0x04

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
DATA15	DATA14	DATA13	DATA12	DATA11	DATA10	DATA9	DATA8
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
DATA7	DATA6	DATA5	DATA4	DATA3	DATA2	DATA1	DATA0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
DATA<15:0>	15:0	Data that is received over the serial interface and stored in the Rx FIFO can be read from this register. A read operation from the spix_rx register will remove the read data from the Rx FIFO.

SoC Motor Driver

spix_status

Reset value = 0x4011

Address Offsets - 0x08

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
RXAF	TXAE	BUSY	-	-	-	-	-
R	R	R	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	RXO	RXF	RXE	TXU	TXO	TXF	TXE
U	R/1C	R	R	R/1C	R/1C	R	R

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
TXE	0	Transmit FIFO is empty. This bit is a read-only bit. 0: Transmit FIFO is not empty 1: Transmit FIFO is empty
TXF	1	Transmit FIFO is full. The transmit FIFO is 16-entries deep. This bit is a read-only bit. 0: Transmit FIFO is not full 1: Transmit FIFO is full
TXO	2	Transmit FIFO overflow. This bit is set when more than 16 messages are written to the transmit FIFO. 0: No overflow occurred 1: Overflow occurred
TXU	3	Transmit FIFO underrun. This bit is set when the module is configured as a slave and the SPI master requests data before the slave prepares the data in the Tx FIFO. The slave data has to be written to the Tx FIFO before the transfer of the word starts. i.e before the master drives the SCK signal. 0: No underrun occurred 1: underrun occurred
RXE	4	Receive FIFO is empty. This bit is a read-only bit. 0: Receive FIFO is not empty 1: Receive FIFO is empty
RXF	5	Receive FIFO is full. The receive FIFO is 16-messages deep. This bit is a read-only bit. 0: Receive FIFO is not full 1: Receive FIFO is full
RXO	6	Receive FIFO overflow. This bit is set when more than 16 messages are received in the receive FIFO. 0: No overflow occurred 1: Overflow occurred

SoC Motor Driver

BUSY	13	Busy bit. This bit indicates whether the SPI interface is busy or idle. In manual chip-select generation mode, the chip-select signal should not be deasserted until this indicates that the interface is idle. This bit is a read-only bit. 0: Idle 1: Busy
TXAE	14	Transmit FIFO is almost empty. This bit is set when the number of messages in the transmit FIFO is less than the programmed threshold value TXAE_THR[4:0] stored in spix_txae_threshold register. This bit is a read-only bit. 0: Transmit FIFO is not almost empty 1: Transmit FIFO is almost empty
RXAF	15	Receive FIFO is almost full. This bit is set when the number of messages in the receive FIFO is greater than the programmed threshold value RXAF_THR[4:0] stored in the spix_rxaf_threshold register. This bit is a read-only bit. 0: Receive FIFO is not almost full 1: Receive FIFO is almost full

SoC Motor Driver

spix_control

Reset value = 0x0000

Address Offsets - 0x0C

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
BPW5	BPW4	BPW3	BPW2	BPW1	BPW0	-	RFIFO
R/W	R/W	R/W	R/W	R/W	R/W	Res	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
RXIE	TXIE	CLKPOL	CLKPHA	BO	WS	SPIx_MODE	SPIx_EN
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
SPIx_EN	0	SPIx module master enable bit. No data will be received or transmitted when the module is disabled. 0: SPIx Disabled (default) 1: SPIx Enabled
SPIx_MODE	1	SPIx mode of operation. 0: Master mode (default) 1: Slave mode
WS	2	Word size. This bit is used only when the BPW bits are equal to 0. 0: 8-bits (default) 1: 16-bits
BO	3	Bit ordering. This bit selects whether the MSb or LSb is trasmitted first. 0: MSb first (default) 1: LSb first
CLKPHA	4	Clock phase configuration. This bit selects the edge on which the module samples data transmitted on the Rx line. 0: Sample on the leading edge (default) 1: Sample on the trailing edge
CLKPOL	5	Clock polarity configuration. This bit selects the normal state of the serial clock SPIx_SCK when the clock is idle. 0: Clock is normally low (default) 1: Clock is normally high
TXIE	6	Transmit completion interrupt enable bit. If enabled, an interrupt is generated when the Tx FIFO is emptied indicating that all messages have been transmitted. 0: Transmit completed interrupt disabled (default) 1: Transmit completed interrupt enabled
RXIE	7	Data received interrupt enable bit. If enabled, an interrupt is generated when the Rx FIFO is not empty, indicating that the module has received some data

SoC Motor Driver

0: Data received interrupt disabled (default)
1: Data received interrupt enabled

RFIFO	8	Reset FIFO buffers. When written with 1, the Tx or Rx FIFO buffers will be cleared. This bit is cleared automatically. 0: Do no reset FIFOs (default) 1: Reset FIFOs
BPW<5:0>	15-10	Bits per word. This number specifies the number of bits that are transmitted or received. When set to 0, the number of bits is determined by the WS bit. Maximum value is 16 bits. 000000: Word size is determined by WS bit (default) 000001: 1-bit word transfer 000010: 2-bit word transfer 000011: 3-bit word transfer 000100: 4-bit word transfer 000101: 5-bit word transfer 000110: 6-bit word transfer 000111: 7-bit word transfer 001000: 8-bit word transfer 001001: 9-bit word transfer 001010: 10-bit word transfer 001011: 11-bit word transfer 001100: 12-bit word transfer 001101: 13-bit word transfer 001110: 14-bit word transfer 001111: 15-bit word transfer 010000: 16-bit word transfer 010001-111111: Unused

SoC Motor Driver

spix_cycles_per_bit

Reset value = 0x0002

Address Offsets - 0x10

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
CPB15	CPB14	CPB13	CPB12	CPB11	CPB10	CPB9	CPB8
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
CPB7	CPB6	CPB5	CPB4	CPB3	CPB2	CPB1	CPB0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
CPB<15:0>	15:0	Cycles per bit. This is a 16-bit unsigned integer that determines for how many clock cycles (FCLK) the transmit clock (SPIX_SCKOUT) is held high or low. In other words, this number acts as a prescaler to the input clock, by which the serial output clock SPIX_SCKOUT is generated when the module is operating as an SPI master. This provides a wide range of transmission frequencies and bit-rates. The minimum value is 2; Meaning that each high and low phase of SCK_OUT is 1 FCLK cycle. Therefore, the resulting total period is the value 2 FCLK clock cycles (value of CPB[15:0]). When CPB[15:0] is set to an odd value, the low phase will last for an extra cycle. This register is not used when the module is operating as a slave.

SoC Motor Driver

spix_txae_threshold

Reset value = 0x0008

Address Offsets - 0x14

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
TXAE_IE	-	-	-	-	-	-	-
R/W	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	TXAE_THR4	TXAE_THR3	TXAE_THR2	TXAE_THR1	TXAE_THR0
U	U	U	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description																		
TXAE_THR <4:0>	4:0	Transmit FIFO almost empty threshold. This number sets the count of used entries in the transmit FIFO, below which the TXAE flag in the spix_status register will be set. This number should always be less than or equal 16. <table><tr><td>00000: 0 entries</td><td>01001: 9 entries</td></tr><tr><td>00001: 1 entry</td><td>01010: 10 entries</td></tr><tr><td>00010: 2 entries</td><td>01011: 11 entries</td></tr><tr><td>00011: 3 entries</td><td>01100: 12 entries</td></tr><tr><td>00100: 4 entries</td><td>01101: 13 entries</td></tr><tr><td>00101: 5 entries</td><td>01110: 14 entries</td></tr><tr><td>00110: 6 entries</td><td>01111: 15 entries</td></tr><tr><td>00111: 7 entries</td><td>10000: 16 entries</td></tr><tr><td>01000: 8 entries (default)</td><td>10001-11111: unused²</td></tr></table>	00000: 0 entries	01001: 9 entries	00001: 1 entry	01010: 10 entries	00010: 2 entries	01011: 11 entries	00011: 3 entries	01100: 12 entries	00100: 4 entries	01101: 13 entries	00101: 5 entries	01110: 14 entries	00110: 6 entries	01111: 15 entries	00111: 7 entries	10000: 16 entries	01000: 8 entries (default)	10001-11111: unused ²
00000: 0 entries	01001: 9 entries																			
00001: 1 entry	01010: 10 entries																			
00010: 2 entries	01011: 11 entries																			
00011: 3 entries	01100: 12 entries																			
00100: 4 entries	01101: 13 entries																			
00101: 5 entries	01110: 14 entries																			
00110: 6 entries	01111: 15 entries																			
00111: 7 entries	10000: 16 entries																			
01000: 8 entries (default)	10001-11111: unused ²																			
TXAE_IE	15	Tx FIFO almost empty interrupt enable bit. When enabled, an interrupt request IRQ_SPIx is generated when the TXAE flag is set in the spix_status register. 0: TXAE interrupt is disabled (default) 1: TXAE interrupt is enabled																		

² The user shall not use a threshold value >16. Otherwise, an undesired interrupt could be generated as the size of the Tx FIFO buffer is 16.

SoC Motor Driver

spix_rxaf_threshold

Reset value = 0x0008

Address Offsets - 0x18

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
RXAF_IE	-	-	-	-	-	-	-
R/W	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	RXAF_THR4	RXAF_THR3	RXAF_THR2	RXAF_THR1	RXAF_THR0
U	U	U	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
RXAF_THR<4:0>	4:0	Receive FIFO almost full threshold. This number sets the count of used entries in the receive FIFO, above which the RXAF flag in the spix_status register will be set. This number should always be less than 16. 00000: 0 entries 01001: 9 entries 00001: 1 entry 01010: 10 entries 00010: 2 entries 01011: 11 entries 00011: 3 entries 01100: 12 entries 00100: 4 entries 01101: 13 entries 00101: 5 entries 01110: 14 entries 00110: 6 entries 01111: 15 entries 00111: 7 entries 1xxxx: unused³ 01000: 8 entries (default)
RXAF_IE	15	Rx FIFO almost full interrupt enable bit. When enabled, an interrupt request IRQ_SPIx is generated when the RXAF flag is set in the spix_status register. 0: RXAF interrupt is disabled (default) 1: RXAF interrupt is enabled

³ The user shall not use a threshold value ≥ 16 . Otherwise, an interrupt will never be generated as the size of the Rx FIFO buffer is 16.

SoC Motor Driver

spix_tx_count

Reset value = 0x0000

Address Offsets - 0x1C

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	TX_COUNT4	TX_COUNT3	TX_COUNT2	TX_COUNT1	TX_COUNT0
U	U	U	R	R	R	R	R

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
TX_COUNT<4:0>	4:0	Transmit FIFO count. This number indicates the count of used entries in the transmit FIFO.
		00000: 0 entries 01001: 9 entries
		00001: 1 entry 01010: 10 entries
		00010: 2 entries 01011: 11 entries
		00011: 3 entries 01100: 12 entries
		00100: 4 entries 01101: 13 entries
		00101: 5 entries 01110: 14 entries
		00110: 6 entries 01111: 15 entries
		00111: 7 entries 10000: 16 entries
		01000: 8 entries 10001-11111: unused ⁴

⁴ Tx FIFO buffer size is 16.

SoC Motor Driver

spix_rx_count

Reset value = 0x0000

Address Offsets - 0x20

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	RX_COUNT4	RX_COUNT3	RX_COUNT2	RX_COUNT1	RX_COUNT0
U	U	U	R	R	R	R	R

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
RX_COUNT<4:0>	4:0	Receive FIFO count. This number indicates the count of used entries in the receive FIFO.
		00000: 0 entries 01001: 9 entries
		00001: 1 entry 01010: 10 entries
		00010: 2 entries 01011: 11 entries
		00011: 3 entries 01100: 12 entries
		00100: 4 entries 01101: 13 entries
		00101: 5 entries 01110: 14 entries
		00110: 6 entries 01111: 15 entries
		00111: 7 entries 10000: 16 entries
		01000: 8 entries 10001-11111: unused ⁵

⁵ Rx FIFO buffer size is 16.

SoC Motor Driver

spix_autotrx_control

Reset value = 0x0000

Address Offsets - 0x24

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	-	-	-	ATR1	ATR0
U	U	U	U	U	U	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
ATR<1:0>	1:0	Auto transmit/receive enable. This feature can be used to eliminate APB accesses to the corresponding FIFOs, when unidirectional transfers are being performed. This feature is useful when the master has completed its transmission, but the slave node is still transmitting data. The master needs to send some data to drive the serial clock SCKOUT. The auto-transmit function (master side) will send dummy data without the need for host access via the APB bus, while the auto-receive function (slave side) will read and discard the dummy data without any processing from the host. In auto-transmit mode, the Tx FIFO is written with the value 0. The number of entries (hence, the number of transmissions) that will be written is specified by the RXTX_COUNT[15:0] bits in the spix_autotrx_count register. In auto-receive mode, the Rx FIFO will be read the number of times specified by the RXTX_COUNT[15:0] bits in the spix_autotrx_count register. The read values will be discarded automatically. 00: Disabled (default) 01: Auto-transmit enabled 10: Auto-receive enabled 11: Unused

SoC Motor Driver

spix_autotrx_count

Reset value = 0x0000

Address Offsets - 0x28

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
RXTX_COUNT15	RXTX_COUNT14	RXTX_COUNT13	RXTX_COUNT12	RXTX_COUNT11	RXTX_COUNT10	RXTX_COUNT9	RXTX_COUNT8
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
RXTX_COUNT7	RXTX_COUNT6	RXTX_COUNT5	RXTX_COUNT4	RXTX_COUNT3	RXTX_COUNT2	RXTX_COUNT1	RXTX_COUNT0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
RXTX_COUNT<15:0>	15:0	Auto transmit/receive count. This number indicates how many times the Tx FIFO will be written with dummy data when the auto-transmit feature is enabled (ATR = 1), or how many times the Rx FIFO will be read when the auto-receive feature is enabled (ATR = 2).

SoC Motor Driver

spix_cs_control

Reset value = 0x0000

Address Offsets - 0x2C

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	-	-	-	CS_MODE	CS_SENSE
U	U	U	U	U	U	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
CS_SENSE	0	Chip-select output sense level. This bit controls the sense level of the chip-select signal SPIx_CSOUT. 0: Active low (default) 1: Active high
CS_MODE	1	Chip-select mode. This bit controls whether the chip-select output is asserted or de-asserted automatically or manually. 0: Automatic (default) 1: Manual Note: These settings only affect SPI master. Chip-select sense level for SPI slave is always active-low.

SoC Motor Driver

spix_cs_assert

Reset value = 0x0001

Address Offsets - 0x30

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	-	-	-	-	CS_ASRT
U	U	U	U	U	U	U	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
CS_ASRT	0	Chip-select assert bit. This bit controls the SPIx_CSOUT output. In automatic chip-select generation mode, this bit should remain enabled. In manual chip-select generation mode, this bit is used to drive the output level on the chip-select output SPIx_CSOUT. 0: De-asserted 1: Asserted (default)

Note, the default value of CS_ASRT after a reset condition is 1.

SoC Motor Driver

spix_cs_assert_bits

Reset value = 0x0001

Address Offsets - 0x34

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
CSA7	CSA6	CSA5	CSA4	CSA3	CSA2	CSA1	CSA0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
CSA<7:0>	7:0	Chip select assert bits. This value determines how many bit periods (i.e. SPIx_SCKOUT cycles) the chip select output SPIx_CSOUT will be asserted for before outputting the clock or data, if the module is configured as a SPI master and automatic chip select control is enabled by setting the CS_MODE bit in the spix_cs_control register to 0. This field is not used when the module is configured as a slave or chip-select is in manual mode (CS_MODE = 1).

SoC Motor Driver

spix_cs_deassert_bits

Reset value = 0x0001

Address Offsets - 0x38

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
CSD7	CSD6	CSD5	CSD4	CSD3	CSD2	CSD1	CSD0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
CSD<7:0>	7:0	Chip select de-assert bits. This value determines how many bit periods (i.e. SPIx_SCKOUT cycles) after the data is transmitted, before the chip select output SPIx_CSOUT will be de-asserted when the module is configured as a SPI master and automatic chip select control is enabled by setting the CS_MODE bit in the spix_cs_control register to 0. This field is not used when the module is configured as a slave or chip-select is in manual mode (CS_MODE = 1).

SoC Motor Driver

spix_cs_idle_bits

Reset value = 0x0001

Address Offsets - 0x3C

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
CSI7	CSI6	CSI5	CSI4	CSI3	CSI2	CSI1	CSI0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
CSI<7:0>	7:0	Chip select idle bits. This value determines the minimum number of bit periods (i.e. SPIx_SCKOUT cycles) after the chip select output SPIx_CSOUT is de-asserted, before which it will be asserted again when the module is configured as a SPI master and automatic chip select control is enabled by setting the CS_MODE bit in the spix_cs_control register to 0. If data does not become available in the Tx FIFO, the actual idle period will exceed this value. This field is not used when the module is configured as a slave or chip-select is in manual mode (CS_MODE = 1).

SoC Motor Driver

1.6.5 Functional Operation

The SPI serial communication is a single-master multiple-slave serial communication interface. The master drives the serial clock and chip-select signals to trigger a transmission. Chip select sense output polarity is configurable for master via the CS_SENSE bit in the spix_cs_control register.

1.6.5.1 Serial clock generation

If the module is configured as the master, the serial clock SPIX_SCKOUT is generated automatically when the transmit register spix_tx is written with data to initiate a transmission. The generated clock frequency is configured via the spix_cycles_per_bit register. The programmed value (CPB[15:0]) determines the duration of the high and low periods in increments of the input clock (FCLK). The minimum programmable value is 2.

If CPB[15:0] is an even value, the duration of the level high and level low of the generated clock SPIX_SCKOUT is equal to half the programmed value. This results in a total period, for each transmitted bit, that is equal to the CPB[15:0]. Figure 1-3 shows the clock generation when the programmed cycles-per-bit value is even.

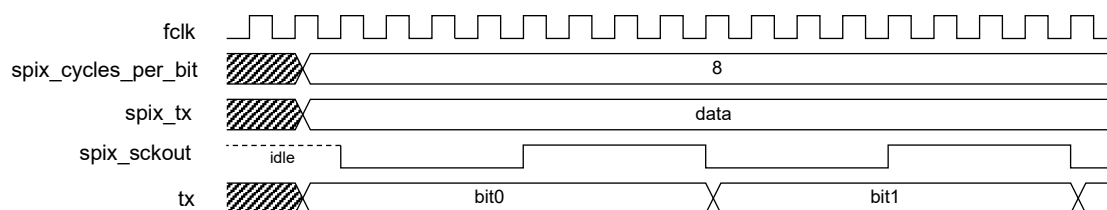


Figure 1-3: Clock generation by SPI master (cycles-per-bit = even value)

If CPB[15:0] is an odd value, the duration of the level high is equal to half of the programmed value (rounded down) and the level low is extended by one clock cycle as shown in Figure 1-4.

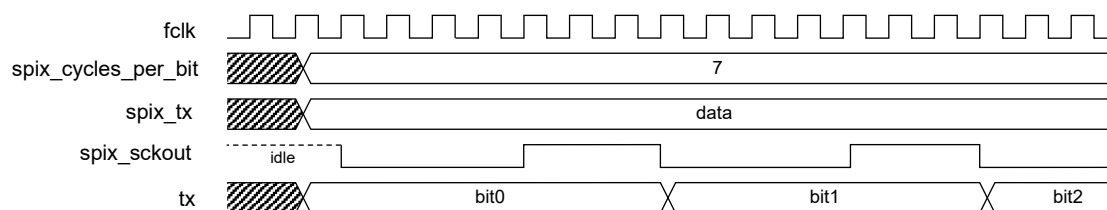


Figure 1-4: Clock generation by SPI master (cycles-per-bit = odd value)

The relation described by Equation (1) translates the CPB[15:0] value to the desired transmission bitrate.

$$CPB[15:0] = \text{floor} \left(\frac{fclk [Hz]}{bitrate [bps]} \right) \quad (1)$$

The SPI module provides programmable parameters for flexible clock configurations. Table 5 and Table 6 summarize the configurable clock parameters and the sampling configurations.

Table 5: Clock configuration parameters

Parameter	Description	Control bit
Clock phase	Sample taken on leading or trailing edge	CLKPHA bit in the spix_control register.
Clock polarity	The normal state of the serial clock when the clock is idle	CLKPOL bit in the spix_control register

SoC Motor Driver

Table 6: Sampling edge configuration

CLKPOL	CLKPHA	Sampling Edge
0	0	Rising edge
	1	Falling edge
1	0	Falling edge
	1	Rising edge

1.6.5.2 Chip-select generation

When the module is configured as an SPI master, the chip-select output is controlled for selecting the slave. The device features a single chip-select output signal that is controlled by hardware. If the SPI interface hosts multiple slaves, additional chip-select signals can be generated via software-controlled GPIO pins.

The SPI module features two modes for chip-select generation: automatic and manual modes. In automatic mode, the chip-select signal SPIx_CSOUT is controlled and generated by hardware. No software intervention is needed during communication to control the generated signal. In manual mode, the chip-select is controlled by software via APB access to a single register.

The module provides flexible chip-select sensing output options for master. The chip-select output can be configured to be active low or active high. This is done by programming the control bit CS_SENSE in the spix_cs_control register.

1.6.5.2.1 Automatic chip-select generation

The automatic chip-select generation functionality enables the module to generate the chip-select output signal SPIx_CSOUT without any processing effort by the host. To select the automatic chip-select feature, the CS_MODE bit in the spix_cs_control register shall be set to 0. The user software shall program the dedicated parameters and the hardware handles signal assertion and de-assertion during transmissions. The CS_ASRT bit in the spix_cs_assert register must be always set to 1 in this mode of operation.

The user software may program the period between the assertion of chip-select and the generation of the serial clock in terms of SPIx_SCKOUT cycles to ensure that the slave is listening and ready for a new transmission. This is done by programming the CSA[7:0] bits in the spix_cs_assert_bits register.

Similarly, the user software may program the period between the last SPIx_SCKOUT clock cycle and the de-assertion of the chip-select. This is done by programming the CSD[7:0] bits in the spix_cs_deassert_bits register.

If multiple transmissions are initiated by the master, the module provides the functionality to program the minimum delay where the chip-select is idle between transmissions, in counts of the SPIx_SCKOUT clock cycle. To configure the idle period in between transmissions, the CSI[7:0] bits in the spix_cs_idle_bits register shall be programmed. The user software must ensure that the transmit data is available in the Tx FIFO before the programmed idle period expires, otherwise the idle time will exceed the programmed value.

Figure 1-5 shows the automatic chip-select generation timing, assuming that the serial clock SPIx_SCKOUT is configured to be normally high when it is idle, and the chip-select output is active low.

SoC Motor Driver

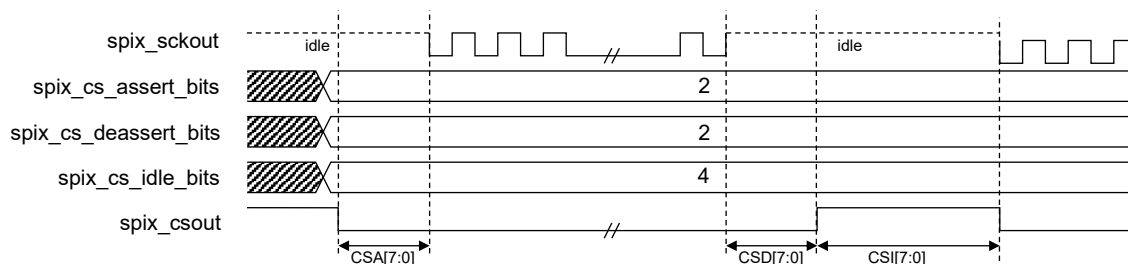


Figure 1-5: Automatic chip-select generation timing diagram

In order to maintain a high throughput, the chip-select is not de-asserted between words in automatic mode, if there are multiple data in the transmit FIFO. If it is desired to de-assert the chip-select between words, the Tx FIFO shall be written with one word at a time, and only re-written when the BUSY flag in the spix_status register indicates that the interface has become idle. The chip-select is automatically de-asserted when a transfer is complete and there is no remaining data in the Tx FIFO buffer.

1.6.5.2.2 Manual chip-select generation

The chip-select output can be controlled manually via software. To enable this feature, the CS_MODE bit in the spix_cs_control register shall be set to 1. In this mode, the spix_cs_assert_bits, spix_cs_deassert_bits and spix_cs_idle_bits registers are not used.

The chip-select output, in manual mode, is controlled via the CS_ASRT bit in the spix_cs_assert register⁶. Depending on the configuration of the CS_SENSE, the chip select output SPIx_CSOUT will vary. Table 7 summarizes the output chip-select level with different configurations in manual chip-select generation mode.

Table 7: Chip-select enable truth table in manual chip-select generation mode

CS_SENSE	CS_ASRT	SPIx_CSOUT level
0	0	HIGH
	1	LOW
1	0	LOW
	1	HIGH

Figure 1-6 shows an example of manual chip-select generation, where the SPIx_SCKOUT clock is configured to be normally high, and the chip-select sense output is active low. Once the software toggles the CS_ASRT bit in the spix_cs_assert register, the output chip-select signal SPIx_CSOUT will be asserted or de-asserted accordingly.

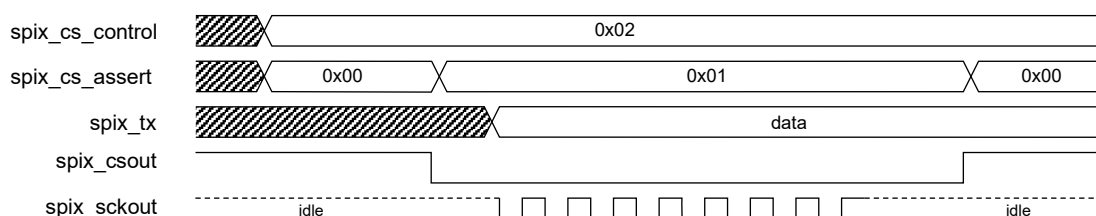


Figure 1-6: Manual chip-select generation mode

⁶ The default value of CS_ASRT after a reset condition is 1.

SoC Motor Driver

1.6.5.3 Transmission

Once the serial clock and chip-select settings are configured, when configured as a master, the module will initiate a transmission whenever the `spix_tx` register is written with data. The written data will be moved to the Tx FIFO and the transmission will commence immediately.

If the module is configured as a slave, the Tx FIFO shall be written with the transmit data before receiving the serial clock signal from the master. Otherwise, the TXU flag is set in the `spix_status` register indicating that the Tx FIFO was empty when the master started driving the serial clock signal.

1.6.5.3.1 Word size and bit ordering

The SPI module provides different bit-size transmissions, supporting from 1-bit up to 16-bit transmissions. The word size is selected using the BPW[5:0] and the WS bits in the `spix_control` register. Both master and slave shall be configured with the same word size for successful transmissions. Table 8 summarizes the allowable different word size configurations.

Table 8: Word size configuration

BPW[5:0]	WS	Transmitted Word Size
0	0	8-bits
	1	16-bits
0 < BPW[5:0] < 17	x	BPW[5:0] bits

Any write operation to the `spix_tx` register will occupy one entry of the Tx FIFO regardless of the word-size. If the written word's size is bigger than the selected transmit word size, the module will discard the extra bits and they will be lost. The module will only transmit the selected number of bits starting from the LSB.

The example shown below in Figure 1-7 shows a 6-bit word transmission. The SPI module is configured as a master (`SPIx_MODE = 0`), the selected word size is 6 bits (`BPW[5:0]=6`). The MSb is transmitted first (`BO=0`), the clock is normally high (`CLKPOL=1`) and the Rx line is sampled on the trailing edge (`CLKPHA=1`). As shown in the figure, once the `spix_tx` register is written with data (8-bits in this case), the transmission will start immediately. The module generates 6 clock cycles and transmits the first 6 bits of the written data only. Any further bits are lost in the process. Note that, even though the MSb is sent first, the module sends the MSb of the amended data.

Figure 1-8 shows the case of sending the same data when the module is configured to transmit the LSb first.

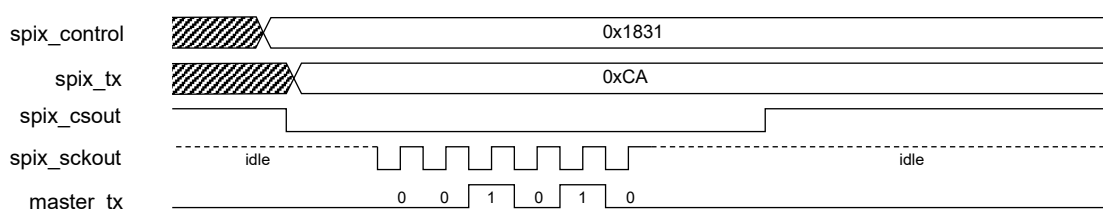


Figure 1-7: 6-bit word transfer - MSb sent first

SoC Motor Driver

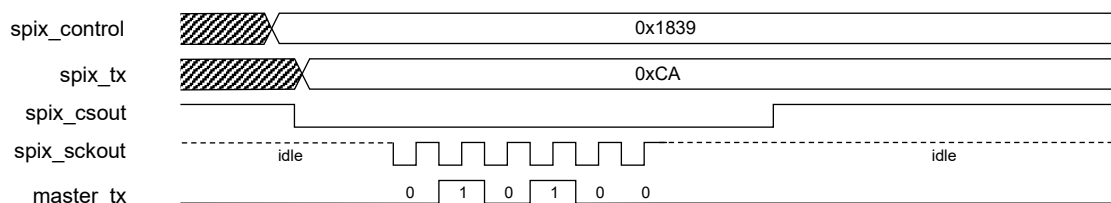


Figure 1-8: 6-bit word transfer- LSb sent first

If the written word size is smaller than the selected transmit word size, the module will append or prepend the transmitted data with zeros, depending on the selected bit ordering (LSb or MSb first respectively).

The example shown below in Figure 1-9 shows a 10-bit word transfer. The SPI module is configured as a master (SPIMODE = 0), the selected word size is 10 bits (BPW[5:0]=10). The MSb is transmitted first (BO=0), the clock is normally low (CLKPOL=0) and the Rx line is sampled on the leading edge (CLKPHA=0). As shown in the figure, once the spix_tx register is written with data (8-bits in this case), the transmission will start immediately. The module generates 10 clock cycles and prepends the transmitted data with two extra bits of the value 0.

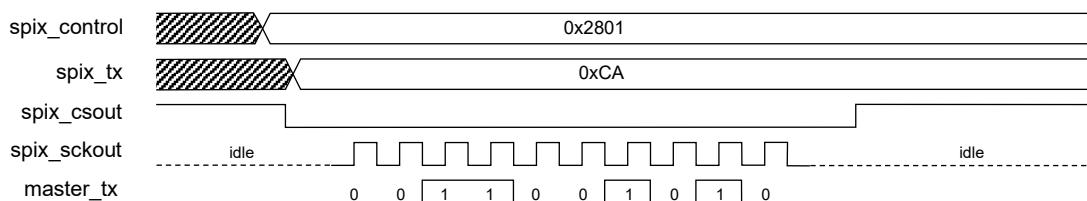


Figure 1-9: 10-bit transmission - data prepending

The same scenario takes place if the selected bit ordering is set to transmit the LSb first (BO=1). The transmitted data is appended with additional 0 bits. Figure 1-10 shows an example of this case.

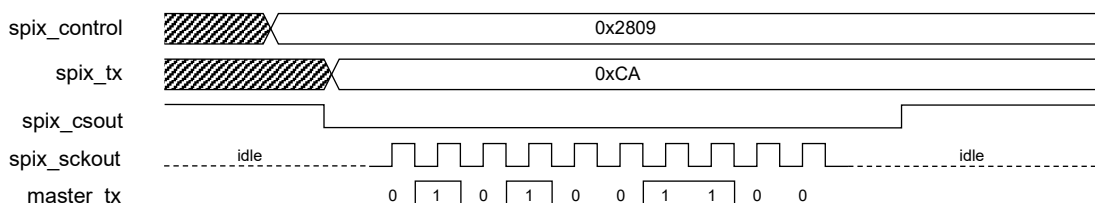


Figure 1-10: 10-bit transmission - data appending

SoC Motor Driver

1.6.5.3.2 Automatic Transmit/Receive functionality

The SPI module features an auto transmit or receive mode dedicated to reduce the processing required from the host in some special scenarios. This mode shall be used when the master needs to read data from the slave, but has run out of data to send. In this case, the master must keep sending dummy data to continue driving the serial clock `SPiX_SCKOUT` in order for the slave to transmit its data.

Typically, the host must continue writing multiple words to the `spix_tx` register to load up the Tx FIFO buffer with enough data needed to drive the clock. Alternatively, the auto transmit or receive feature provides a simpler method to automatically program the module to complete this task without any further instructions from the host; The auto-transmit feature shall be enabled in the SPI master, by setting `ATR = 1` in the `spix_autotrx_control` register. The master will automatically load the Tx FIFO buffer with zeros. The number of times the Tx FIFO is written, i.e. the number of transmitted words⁷, is defined by the `spix_autotrx_count` register. The user software shall program this number before enabling the auto-transmit feature.

The auto-receive feature shall be enabled in the SPI slave when it is known that the incoming data on the rx line is dummy data. The feature is enabled by setting `ATR = 2` in the `spix_autotrx_control` register. Once enabled, the module will automatically read the Rx FIFO and discard the data. Hence, relieving the host from the need to read the Rx FIFO repeatedly and discarding the data manually, saving some processing effort. The number of times the Rx FIFO is read is defined by the `spix_autotrx_count` register. The user software shall program this number before enabling the auto-receive feature.

Figure 1-11 demonstrates an example of the SPI master auto-transmit feature. The SPI module is configured as an SPI master, with 3-bit wide words. The auto-transmit count is set to 3, meaning that 3 dummy words will be transmitted and the clock is generated for 9 bit times (3x 3-bit words). Once the auto-transmit feature is enabled by writing 1 to the `spix_autotrx_control` register, the transmission begins immediately. The chip-select output is asserted and the serial clock is driven. The master transmits zeros on the TX line. The slave is able to drive the master's RX line with the required data. The read data is loaded into the master's Rx FIFO buffer and can be read by the host when desired.

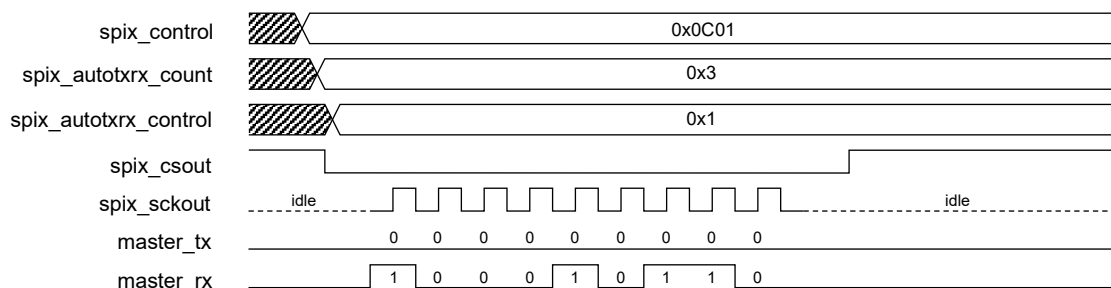


Figure 1-11: SPI master auto-transmit feature

⁷ The word size, i.e. the number of transmitted bits, is defined by the `BPW[5:0]` and `WS` bits as explained in section 1.6.5.3.1.

SoC Motor Driver

1.6.5.4 Interrupt Management

Each SPI channel generates a single active-high interrupt request signal IRQ_SPIx. Four interrupt sources will generate the interrupt request signal. Each interrupt source has an independent enable bit for controlling the interrupt request generation as desired. Interrupt sources can be polled via the spix_status register if the interrupt generation is disabled. Table 9 summarizes all interrupt sources and their corresponding control and status bits.

Table 9: SPI interrupt sources

Interrupt Source	Description	Status bit	Enable bit
Transmit interrupt	An interrupt is raised when the transmit buffer is empty and the TXIE flag in the spix_control register is set to 1. This indicates that the transmitter has no further data to transmit	TXE bit in the spix_status register	TXIE bit in the spix_control register
Transmit FIFO almost empty interrupt	An interrupt is raised when the transmit FIFO is almost empty (when the number of entries is less than the programmed threshold defined in the spix_txae_threshold register) and the TXAE_IE flag in the spix_txae_threshold register is set to 1	TXAE bit in the spix_status register	TXAE_IE bit in the spix_txae_threshold register
Receive interrupt	An interrupt is raised when the receive FIFO buffer is not empty and the RXIE flag in the spix_control register is set to 1. This indicates that some data has been received	RXE bit in the spix_status register	RXIE bit in the spix_control register
Receive FIFO almost full interrupt	An interrupt is raised when the receive FIFO buffer is almost full (when the number of entries is greater than the programmed threshold defined in the spix_rxaf_threshold register) and the RXAF_IE flag in the spix_rxaf_threshold register is set to 1	RXAF bit in the spix_status register	RXAF_IE bit in the spix_rxaf_threshold register

Figure 1-12 shows the interrupt request generation logic for a given SPI channel. The displayed logic is repeated for both channels, resulting in two distinct interrupt request signals.

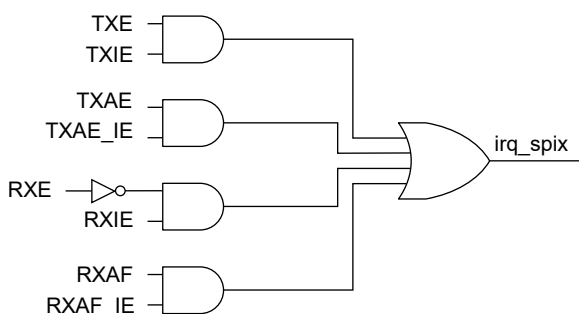


Figure 1-12: Interrupt request generation logic of a given SPI channel

SoC Motor Driver

1.7 Universal Asynchronous Receiver Transmitter (UART)

1.7.1 Features

- Configurable word size (7 or 8 data bits)
- Configurable number of stop bits (1 or 2 stop bits)
- Configurable parity bit settings (None/Even/Odd parity)
- Programmable bit rate
- 16-message deep Rx FIFO
- 16-message deep Tx FIFO
- DMA flow-control interface
- Configurable interrupt request sources

1.7.2 Introduction

The device features a UART module with two independent channels for asynchronous serial communications. The module has 16-deep Rx and Tx FIFO buffers, supports configurable word sizes, number of stop bits and parity settings. The module also supports a wide range of bitrates. Finally, the UART module hosts a DMA flow-control interface to support DMA transfers.

1.7.3 Functional Description

The device features two independent UART channels that can be used for asynchronous serial communications.

The UART shares the same bus address with the SPI module as both modules are considered mutually exclusive. Meaning that for a given SCI channel, it could operate as either a UART or SPI block in any given time. Configuring the behavior of an SCI channel is set by the `sysctrl_ctrl` register in the System Control Unit.

The transmission baudrate is programmable via the `uartx_cycles_per_bit` register. This allows a broad range of bitrates to fit any user application.

The parity, stop bit and data width settings are also programmable via the `uartx_control` register.

The Tx and Rx FSM will handle all transmissions at bit-level and update the `uartx_status` register accordingly. Once the Tx FIFO is updated with data, data transmission is commenced. Similarly, the Rx FIFO is filled with received data upon data reception.

Once a status bit is updated by the Tx or Rx FSM, the interrupt management logic will generate an IRQ, if the option is enabled by the user. Each UART channel has a single dedicated active-high IRQ signal connected to the Cortex M4's NVIC.

Each UART channel also hosts a DMA flow controller which handles the handshake protocol with the DMA Controller for DMA transfer requests.

Figure 1-13 shows the general block diagram of a given UART channel.

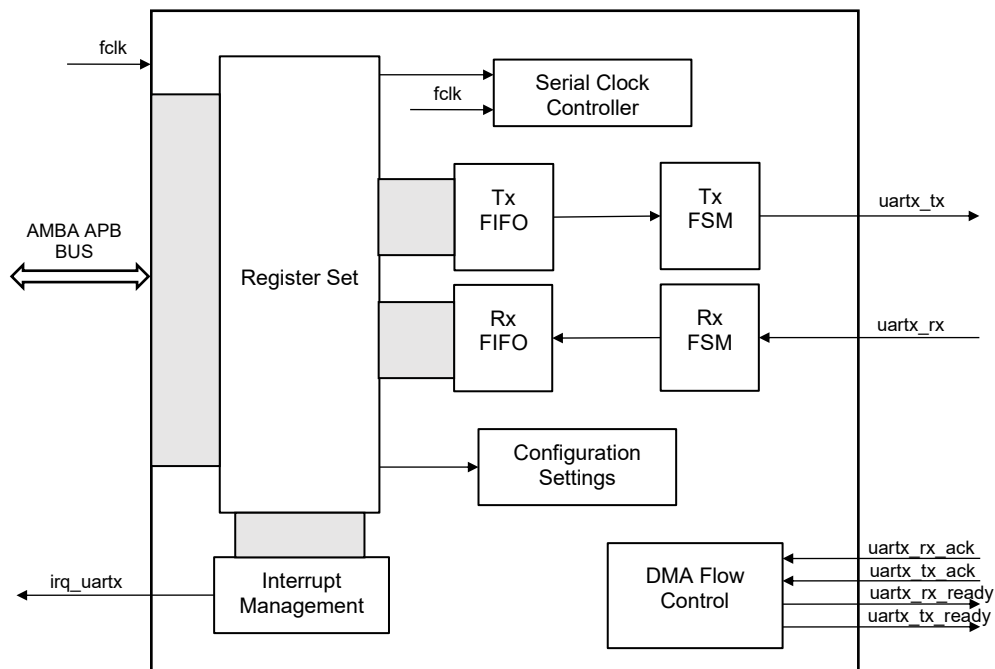


Figure 1-13: General block diagram of a UART channel

1.7.4 Register Set

The UART register set is accessible through the AMBA APB Bus interface. Registers are word-aligned. Reserved and unimplemented bits are read as zero.

The register map consists of 10 16-bit registers. Each channel has its dedicated register set. Table 10 shows the register map of UART channels 0 and 1 and summarizes the registers' names, access type, address offset and reset values.

UART0 Base Address: 0x4000A000

UART1 Base Address: 0x4000B000

Table 10: UART register map

Register Name	Access Type	Address Offset	Reset Value
UART Base Address: 0x4000A000			
uart0 tx	W	0x00	0x0000
uart0 rx	R	0x04	0x0000
uart0 status	R/1C	0x08	0x4009
uart0 control	R/W	0x0C	0x0000
uart0 cycles per bit	R/W	0x10	0x015B
uart0 txae threshold	R/W	0x18	0x0008
uart0 rxaf threshold	R/W	0x1C	0x0008
uart0 tx count	R	0x30	0x0000
uart0 rx count	R	0x34	0x0000
uart0 tx guard	R/W	0x38	0x0000
UART Base Address: 0x4000B000			
uart1 tx	W	0x00	0x0000
uart1 rx	R	0x04	0x0000
uart1 status	R/1C	0x08	0x4009
uart1 control	R/W	0x0C	0x0000
uart1 cycles per bit	R/W	0x10	0x015B
uart1 txae threshold	R/W	0x18	0x0008
uart1 rxaf threshold	R/W	0x1C	0x0008
uart1 tx count	R	0x30	0x0000
uart1 rx count	R	0x34	0x0000
uart1 tx guard	R/W	0x38	0x0000

uartx_tx

Reset value = 0x0000

Address Offsets - 0x00

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
DATA7	DATA6	DATA5	DATA4	DATA3	DATA2	DATA1	DATA0
W	W	W	W	W	W	W	W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
DATA<7:0>	7:0	Data to be transmitted over the serial interface. Data written to this register will be written to the Tx FIFO. To avoid any data loss, this register should not be written to if the TXF bit in the uartx_status register is set.

uartx_rx

Reset value = 0x0000

Address Offsets - 0x04

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
DATA7	DATA6	DATA5	DATA4	DATA3	DATA2	DATA1	DATA0
R	R	R	R	R	R	R	R

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
DATA<7:0>	7:0	Data that is received over the serial interface and stored in the Rx FIFO can be read from this register. A read operation from the spix_rx register will remove the read data from the Rx FIFO.

uartx_status

Reset value = 0x4009

Address Offsets - 0x08

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
RXAF	TXAE	-	-	-	-	PE	FE
R	R	Res	Res	Res	Res	R/1C	R/1C

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
RXB	RXP	RXO ⁸	RXF	RXE	TXO ⁸	TXF	TXE
R/1C	R	R/1C	R	R	R/1C	R	R

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
TXE	0	Transmit FIFO is empty. This bit is a read-only bit. 0: Transmit FIFO is not empty 1: Transmit FIFO is empty
TXF	1	Transmit FIFO is full. The transmit FIFO is 16-entries deep. This bit is a read-only bit. 0: Transmit FIFO is not full 1: Transmit FIFO is full
TXO ⁸	2	Transmit FIFO overflow. This bit is set when more than 16 messages are written to the transmit FIFO. Writing '1' to this bit location clears it. 0: No overflow occurred 1: Overflow occurred
RXE	3	Receive FIFO is empty. This bit is a read-only bit. 0: Receive FIFO is not empty 1: Receive FIFO is empty
RXF	4	Receive FIFO is full. The receive FIFO is 16-messages deep. This bit is a read-only bit. 0: Receive FIFO is not full 1: Receive FIFO is full
RXO ⁸	5	Receive FIFO overflow. This bit is set when more than 16 messages are received in the receive FIFO. Writing '1' to this bit location clears it. 0: No overflow occurred 1: Overflow occurred
RXP	6	Received parity bit. This bit displays the value of the parity bit of the data at the front of the FIFO. This bit should be read before the corresponding data is read from the Rx FIFO. This bit is a read-only bit. 0: Parity bit was 0 1: Parity bit was 1
RXB	7	Received break. The Rx FIFO is not updated with data when a break is received. Writing '1' to this bit location clears it.

⁸ If an overflow occurs, the contents of the corresponding FIFO are undefined and the FIFO should be reset by setting the RF bit in the uartx_control register.

0: Break is not received
 1: Break is received

FE	8	Framing error. This bit is set when a framing error is detected, i.e. a fault is detected at the start and stop bit-positions. The received data is not written to the Rx FIFO when a framing error is detected. Writing '1' to this bit location clears it. 0: No framing errors detected 1: Framing error detected
PE	9	Parity error. This bit is set when a parity error is detected. A parity error is detected when the received parity bit does not match the calculated parity. Writing '1' to this bit location clears it. 0: No parity errors detected 1: Parity error detected
TXAE	14	Transmit FIFO is almost empty. This bit is set when the number of messages in the transmit FIFO is less than the programmed threshold value TXAE_THR[4:0] stored in uartx_txae_threshold register. This bit is a read-only bit. 0: Transmit FIFO is not almost empty 1: Transmit FIFO is almost empty
RXAF	15	Receive FIFO is almost full. This bit is set when the number of messages in the receive FIFO is greater than the programmed threshold value RXAF_THR[4:0] stored in the uartx_rxaf_threshold register. This bit is a read-only bit. 0: Receive FIFO is not almost full 1: Receive FIFO is almost full

uartx_control

Reset value = 0x0000

Address Offsets - 0x0C

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	RF	-	-	RXBIE	RXIE	TXIE
Res	Res	R/W	Res	Res	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
TXB	-	PB2	PB1	PB0	DB	SB	UARTx_EN
R/W	Res	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
UARTx_EN	0	UARTx module master enable bit. No data will be received or transmitted when the module is disabled. 0: UARTx Disabled (default) 1: UARTx Enabled
SB	1	Number of stop bits at the end of each transmitted byte. 0: 1 stop bit (default) 1: 2 stop bits
DB	2	Number of data bits per each transmitted data byte. 0: 8-bits (default) 1: 7-bits
PB<2:0>	5:3	Parity bit control. 000: No parity bit (default) 001: Even parity 010: Odd parity 011: Mark 100: Space 101-111: Reserved
TXB	7	Transmit break. When this bit is set, the transmit data line will be held low, signaling a break. The break will be signalled until this bit is cleared. 0: Do not send a break (default) 1: Send a break
TXIE	8	Transmit completion interrupt enable bit. If enabled, an interrupt is generated when the Tx FIFO is emptied indicating that all messages have been transmitted. 0: Transmit completed interrupt disabled (default) 1: Transmit completed interrupt enabled
RXIE	9	Data received interrupt enable bit. If enabled, an interrupt is generated when the Rx FIFO is not empty, indicating that the module has received some data 0: Data received interrupt disabled (default) 1: Data received interrupt enabled
RXBIE	10	Break received interrupt enable. If enabled, an interrupt request is generated when the module receives a break.

		0: Break received interrupt generation is disabled (default) 1: Break received interrupt generation is enabled
RF	13	Reset FIFOs. When written with 1, transmit and receive FIFO buffers will be cleared. This bit will be cleared automatically after the FIFO buffers are cleared. 0: Do not reset FIFO buffers 1: Reset FIFO buffers

uartx_cycles_per_bit

Reset value = 0x015B

Address Offsets - 0x10

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
CPB15	CPB14	CPB13	CPB12	CPB11	CPB10	CPB9	CPB8
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
CPB7	CPB6	CPB5	CPB4	CPB3	CPB2	CPB1	CPB0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
CPB<15:0>	15:0	Cycles-per-bit. This is a 16-bit unsigned integer that determines for how many clock cycles (FCLK) each bit is transmitted for. The number of cycles is CPB[15:0] + 1. Equation (1) shows the relation between the desired baudrate and CPB[15:0]. The use of a 16-bit number provides support for a wide range of clock frequencies and baudrates. The value 0 is not supported.

$$CPB[15:0] = \left\lfloor \frac{f_{clk} [Hz]}{bitrate [bps]} \right\rfloor - 1 \quad (2)$$

uartx_txae_threshold

Reset value = 0x0008

Address Offsets - 0x18

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
TXAE_IE	-	-	-	-	-	-	-
R/W	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	TXAE_THR4	TXAE_THR3	TXAE_THR2	TXAE_THR1	TXAE_THR0
U	U	U	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
TXAE_THR <4:0>	4:0	Transmit FIFO almost empty threshold. This number sets the count of used entries in the transmit FIFO, below which the TXAE flag in the uartx_status register will be set. This number should always be less than or equal 16. 00000: 0 entries 01001: 9 entries 00001: 1 entry 01010: 10 entries 00010: 2 entries 01011: 11 entries 00011: 3 entries 01100: 12 entries 00100: 4 entries 01101: 13 entries 00101: 5 entries 01110: 14 entries 00110: 6 entries 01111: 15 entries 00111: 7 entries 10000: 16 entries 01000: 8 entries (default) 10001-11111: unused ⁹
TXAE_IE	15	Tx FIFO almost empty interrupt enable bit. When enabled, an interrupt request IRQ_UARTx is generated when the TXAE flag is set in the uartx_status register. 0: TXAE interrupt is disabled (default) 1: TXAE interrupt is enabled

⁹ The user shall not use a threshold value >16. Otherwise, an undesired interrupt could be generated as the size of the Tx FIFO buffer is 16.

uartx_rxaf_threshold

Reset value = 0x0008

Address Offsets - 0x1C

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
RXAF_IE	-	-	-	-	-	-	-
R/W	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	RXAF_THR4	RXAF_THR3	RXAF_THR2	RXAF_THR1	RXAF_THR0
U	U	U	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
RXAF_THR<4:0>	4:0	Receive FIFO almost full threshold. This number sets the count of used entries in the receive FIFO, above which the RXAF flag in the uartx_status register will be set. This number should always be less than 16. 00000: 0 entries 00001: 1 entry 00010: 2 entries 00011: 3 entries 00100: 4 entries 00101: 5 entries 00110: 6 entries 00111: 7 entries 01000: 8 entries (default) 01001: 9 entries 01010: 10 entries 01011: 11 entries 01100: 12 entries 01101: 13 entries 01110: 14 entries 01111: 15 entries 1xxxx: unused ¹⁰
RXAF_IE	15	Rx FIFO almost full interrupt enable bit. When enabled, an interrupt request IRQ_UARTx is generated when the RXAF flag is set in the uartx_status register. 0: RXAF interrupt is disabled (default) 1: RXAF interrupt is enabled

¹⁰ The user shall not use a threshold value ≥ 16 . Otherwise, an interrupt will never be generated as the size of the Rx FIFO buffer is 16.

uartx_tx_count

Reset value = 0x0000

Address Offsets - 0x30

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	TX_COUNT4	TX_COUNT3	TX_COUNT2	TX_COUNT1	TX_COUNT0
U	U	U	R	R	R	R	R

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
TX_COUNT<4:0>	4:0	Transmit FIFO count. This number indicates the count of used entries in the transmit FIFO. 00000: 0 entries 00001: 1 entry 00010: 2 entries 00011: 3 entries 00100: 4 entries 00101: 5 entries 00110: 6 entries 00111: 7 entries 01000: 8 entries 01001: 9 entries 01010: 10 entries 01011: 11 entries 01100: 12 entries 01101: 13 entries 01110: 14 entries 01111: 15 entries 10000: 16 entries 10001-11111: unused¹¹

¹¹ Tx FIFO buffer size is 16.

uartx_rx_count

Reset value = 0x0000

Address Offsets - 0x34

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	RX_COUNT4	RX_COUNT3	RX_COUNT2	RX_COUNT1	RX_COUNT0
U	U	U	R	R	R	R	R

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
RX_COUNT<4:0>	4:0	Receive FIFO count. This number indicates the count of used entries in the receive FIFO. 00000: 0 entries 00001: 1 entry 00010: 2 entries 00011: 3 entries 00100: 4 entries 00101: 5 entries 00110: 6 entries 00111: 7 entries 01000: 8 entries 01001: 9 entries 01010: 10 entries 01011: 11 entries 01100: 12 entries 01101: 13 entries 01110: 14 entries 01111: 15 entries 10000: 16 entries 10001-11111: unused¹²

¹² Rx FIFO buffer size is 16.

uartx_tx_guard

Reset value = 0x0000

Address Offsets - 0x38

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	TG8
U	U	U	U	U	U	U	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
TG7	TG6	TG5	TG4	TG3	TG2	TG1	TG0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read

W: Write

1C: Write 1 to clear. Writing 0 has no effect

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
TG<8:0>	8:0	Transmit guard. The transmit guard field specifies the number of bit periods between each data transmission. The number of bit periods is TG[8:0] + 1. 00000000: Guard time disabled 00000001: 2 bit-periods 00000010: 3 bit-periods ... 11111110: 511 bit-periods 11111111: 512 bit-periods

1.7.5 Functional Operation

The UART serial communication is a two-wire serial communication interface. The transmitter and receiver work in an asynchronous environment. The data is transmitted in a byte format, where the transmitted byte is framed between a start bit and a stop bit. An optional parity bit is added to ensure the sanity of the transmitted byte.

1.7.5.1 Baud-rate setting

The UART serial interface comprises two wires for communication: Tx and Rx. The UART communication does not need a serial clock to be transmitted between nodes, and thus does not include a master-slave system. To guarantee a stable communication, the receiver and transmitter should both be configured to operate at the same bitrate/baudrate.

The UART module supports a wide range of bitrates, as it utilizes a 16-bit number CPB[15:0] stored in the `uartx_cycles_per_bit` register, which translates the desired bitrate to the internal clock signal FCLK. The relation given by Equation (1) shall be used to program the correct value of CPB[15:0] with respect to the desired bitrate. The number of cycles per bit is equal to CPB[15:0] + 1.

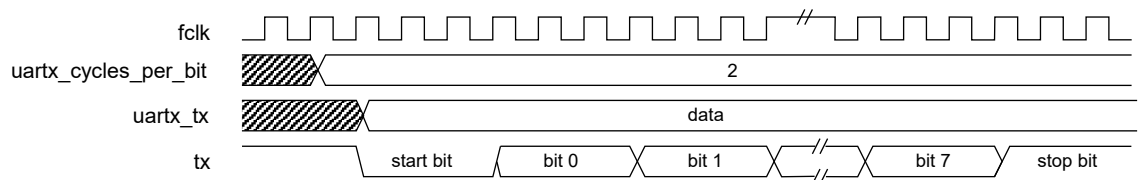


Figure 1-14: Cycles per bit configuration

1.7.5.2 Parity setting

The UART module supports different parity configurations. The user may select the use of an even parity, odd parity, mark or space in the parity bit position. If any of these options is selected, The corresponding parity bit will be added at the end of the data bits, i.e. between the last data bit and the stop bits, during transmission, and will be processed accordingly during the reception of a frame.

The parity bit setting is configured using the PB[2:0] bits in the `uartx_control` register. Table 11 summarizes the parity bit selection settings.

Table 11: Parity bit selection bits

PB[2:0]	Parity bit
0	No parity bit
1	Even parity
2	Odd parity
3	Mark
4	Space

Figure 1-15 shows an example of an even parity bit transmission. Initially, the module is configured to add no parity bits to the transmitted byte (PB[2:0]=0). The transmitted data is followed by the stop bit immediately after the transmission of the last data bit. When the parity bit is set to even parity (PB[2:0] = 1), an extra bit is added after the last bit at the subsequent transmission. The parity bit, in this case '0', is added to maintain the number of transmitted '1' bits even. The parity bit is followed by a stop bit.

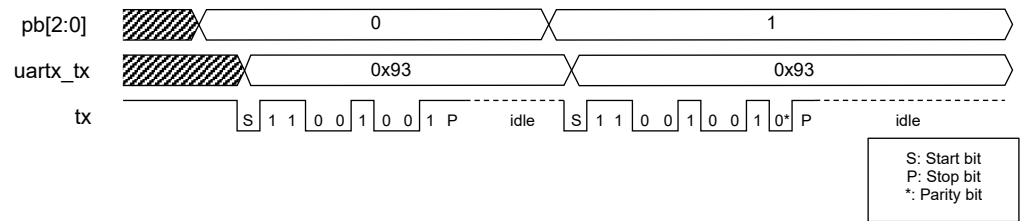


Figure 1-15: Even parity bit example

1.7.5.3 Stop-bit setting

The number of stop bits at the end of the transmission of each byte is programmable via the SB bit in the `uartx_control` register. The module supports 1 or 2 stop bits to be transmitted or processed at the end of each byte.

Figure 1-16 shows an example of varying the number of stop bits at the end of any transmitted byte.

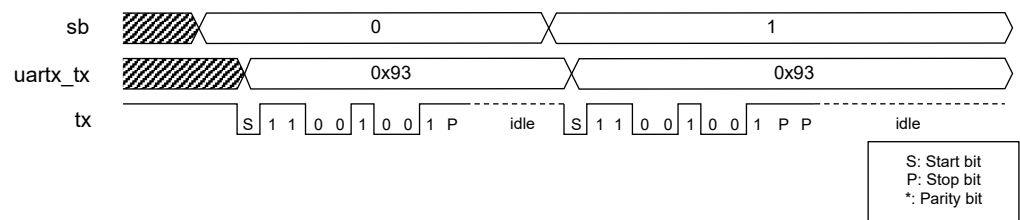


Figure 1-16: Different stop-bit configurations

1.7.5.4 Number of data bits

The number of transmitted data bits is programmable via the DB bit in the `uartx_control` register. The UART module supports the transmission and reception of either 7 or 8 data bits in a single transmission. In case a 7-bit transmission is selected, the MSb is ignored and only the 7 LSB are transmitted when the `uartx_tx` register is written with data.

Figure 1-17 shows an example of varying the number of transmitted bits.

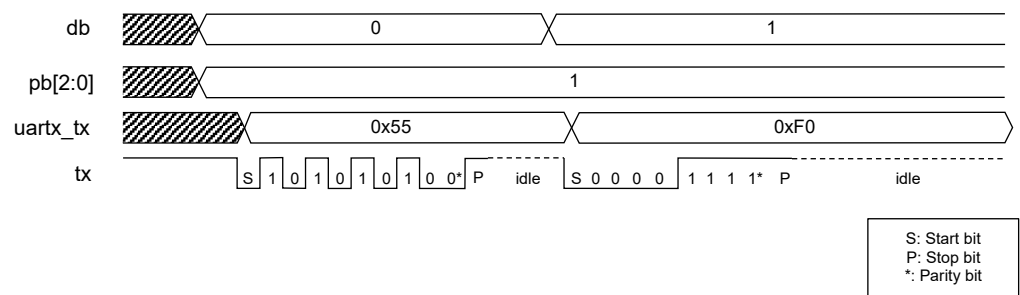


Figure 1-17: Different number of data bits

1.7.5.5 Transmission break

The UART module supports the transmission and detection of a break. The break transmission is controlled using the TXB bit in the `uartx_control` register. Once the bit is set, the Tx line will be held low, signalling a break. The line remains held until the TXB bit is cleared.

Figure 1-18 shows an example of a break transmission. The Tx line is held low until the TXB bit is cleared. Writing to `uart_tx` register while TXB is set has no effect.

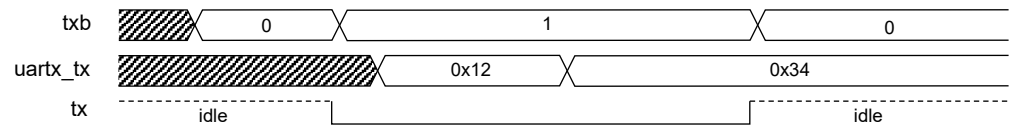


Figure 1-18: Break transmission example

1.7.5.6 Transmission guard

The UART module implements a guard feature that controls the number of bit periods that are inserted between each data transmission. The guard setting is programmable via the `uartx_tx_guard` register. The Tx line remains held high for a duration of $TG[8:0] + 1$ bit periods if the transmission guard insertion is enabled¹³.

Figure 1-19 shows an example of multiple transmission guard settings. Initially, the transmission guard is disabled as $TG[8:0]$ remains 0. Therefore, no guard time is inserted between the transmission of the first two bytes. When $TG[8:0] = 2$, a guard time of 3 bit periods is inserted between the second and third bytes as shown in the figure. Finally, a guard time of 4 bit periods is inserted between the last two bytes as $TG[8:0]$ is set to 3. The Tx line remains high during the guard time period.

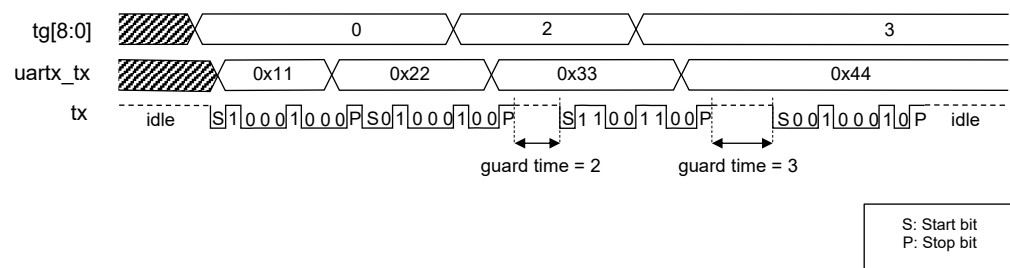


Figure 1-19: Transmission guard example

¹³ Transmission guard is enabled when $TG[8:0] > 0$.

1.7.6 Interrupt management

Each UART channel has its dedicated active-high interrupt request signal IRQ_UARTx connected to the Cortex M4's NVIC.

Each channel has 5 distinct interrupt request sources, each having its dedicated enable bit, to control the interrupt request generation. For polling based systems, the `uartx_status` register provides all the information concerning the status of the module.

Table 12 summarizes all interrupt sources with their corresponding status and enable bits.

Table 12: UART interrupt sources

Interrupt Source	Description	Status bit	Enable bit
Transmit empty	An interrupt will be flagged when the transmit FIFO is empty and the TXIE flag in the <code>uartx_control</code> register is enabled. This indicates that all data has been transmitted.	TXE bit in the <code>uartx_status</code> register	TXIE bit in the <code>uartx_control</code> register
Transmit FIFO almost empty	An interrupt will be flagged when the transmit FIFO is almost empty and the TXAE flag is enabled. The almost empty condition is true when the number of elements in the transmit FIFO is less than the programmed threshold value TXAE_THR[4:0] stored in the <code>uartx_txae_threshold</code> register.	TXAE bit in the <code>uartx_status</code> register.	TXAE_IE bit in the <code>uartx_txae_threshold</code> register.
Message received	An interrupt will be flagged when the receive FIFO is not empty and the RXIE flag in the <code>uartx_control</code> register is enabled. This indicates that a message has been received.	RXE bit in the <code>uartx_status</code> register.	RXIE bit in the <code>uartx_control</code> register.
Receive FIFO almost full	An interrupt will be flagged when the receive FIFO is almost full and the RXAF_IE flag is enabled. The almost full condition is true when the number of elements in the receive FIFO is more than the programmed threshold value RXAF_THR[4:0] in the <code>uartx_rxaf_threshold</code> register.	RXAF bit in the <code>uartx_status</code> register.	RXAF_IE bit in the <code>uartx_rxaf_threshold</code> register.
Break received	An interrupt will be flagged when a break is detected on the Rx line and the RXBIE flag in the <code>uartx_control</code> register is enabled.	RXB bit in the <code>uartx_status</code> register.	RXBIE bit in the <code>uartx_control</code> register.

Figure 1-20 shows the interrupt request generation logic of a given UART channel. The displayed logic is repeated for both UART channels, outputting two independent interrupt request signals.

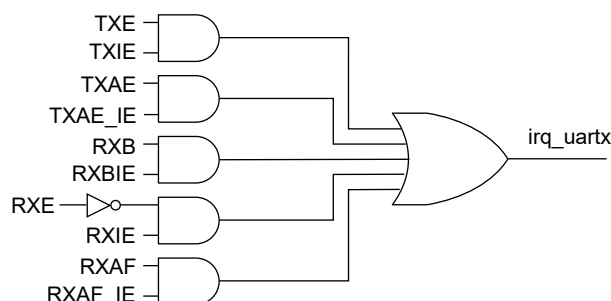


Figure 1-20: Interrupt request generation logic of a given UART channel

Revision Control:

Dec 10/2020	Initial Revision
Nov 11/2021	Logo Updated Added revision control
April 12/2023	Minor revisions for typos and readability Corrected UART links in the register tables Renamed spix_cs_enable register to spix_cs_assert, and cs_en bit to cs_asrt
Feb 19/2025	Corrected description for spix_cs_control Register to state the chip-select sense level is only configurable for SPI master.
Feb 20/2026	Changed transmit register to write only

Copyright ©2026, Allegro MicroSystems, Inc

Allegro MicroSystems, Inc reserves the right to make, from time to time, such departures from the detail specifications as may be required to permit improvements in the performance, reliability, or manufacturability of its products. Before placing an order, the user is cautioned to verify that the information being relied upon is current.

Allegro's products are not to be used in life support devices or systems, if a failure of an Allegro product can reasonably be expected to cause the failure of that life support device or system, or to affect the safety or effectiveness of that device or system.

The information included herein is believed to be accurate and reliable. However, Allegro MicroSystems, Inc assumes no responsibility for its use; nor for any infringement of patents or other rights of third parties which may result from its use.